OOP PRACTICAL SOLUTIONS

All 22 C++ programs from the NMIET OOP Practical Program list

Exam-friendly version

Each solution is written as a separate, complete C++ program. Compile and run one program at a time. The explanations briefly identify the main OOP concept used.

Important: For practicals where the question leaves a detail unspecified, the assumption is stated below that program (for example, two subjects out of 100 in percentage questions).

1. Student details (roll number, name and marks)

```
#include <iostream>
using namespace std;

class Student {
    int rollNo;
    string name;
    float marks;
public:
    void accept() {
        cout << "Enter roll number, name and marks: ";
        cin >> rollNo >> name >> marks;
    }
    void display() {
        cout << "\nRoll No: " << rollNo;
        cout << "\nName: " << name;
        cout << "\nMarks: " << marks << endl;
    }
};

int main() {
    Student s;
    s.accept();
    s.display();
    return 0;
}
```

Explanation: A Student class stores the three details. The accept() function takes input and display() prints it.

2. Student percentage (two subjects, one object)

```
#include <iostream>
using namespace std;

class Student {
    int rollNo;
    string name;
    float m1, m2, percentage;
public:
    void accept() {
        cout << "Enter roll number and name: ";
        cin >> rollNo >> name;
        cout << "Enter marks of two subjects: ";
        cin >> m1 >> m2;
    }
    void calculate() {
        percentage = (m1 + m2) / 2.0;
    }
    void display() {
        cout << "\nRoll No: " << rollNo;
        cout << "\nName: " << name;
        cout << "\nSubject 1: " << m1;
        cout << "\nSubject 2: " << m2;
        cout << "\nPercentage: " << percentage << "%\n";
    }
};

int main() {
    Student s;
    s.accept();
    s.calculate();
    s.display();
    return 0;
}
```

Explanation: Assuming each subject is out of 100, percentage = (m1 + m2) / 2.

3. Employee details (one object)

```
#include <iostream>
using namespace std;

class Employee {
    int empId;
    string name;
    float salary;
public:
    void accept() {
```

```
        cout << "Enter employee ID, name and salary: ";
        cin >> empId >> name >> salary;
    }
    void display() {
        cout << "\nEmployee ID: " << empId;
        cout << "\nName: " << name;
        cout << "\nSalary: " << salary << endl;
    }
};

int main() {
    Employee e;
    e.accept();
    e.display();
    return 0;
}
```

Explanation: The object e calls the member functions to accept and display employee information.

4. Display employees whose salary is greater than 5000 (five objects)

```
#include <iostream>
using namespace std;

class Employee {
    string empName;
    int empId;
    float salary;
public:
    void accept() {
        cout << "Enter employee name, ID and salary: ";
        cin >> empName >> empId >> salary;
    }
    void display() {
        cout << "\nName: " << empName
             << "\nID: " << empId
             << "\nSalary: " << salary << "\n";
    }
    bool isEligible() {
        return salary > 5000;
    }
};

int main() {
    Employee e[5];

    for (int i = 0; i < 5; i++) {
        cout << "\nEmployee " << i + 1 << endl;
        e[i].accept();
    }

    cout << "\nEmployees with salary greater than 5000:\n";
    for (int i = 0; i < 5; i++) {
        if (e[i].isEligible())
            e[i].display();
    }
    return 0;
}
```

Explanation: An array stores five Employee objects. Only objects with salary > 5000 are displayed.

5. Area of rectangle

```
#include <iostream>
using namespace std;

class Rectangle {
    float length, breadth;
public:
    void accept() {
        cout << "Enter length and breadth: ";
        cin >> length >> breadth;
    }
    void displayArea() {
        cout << "Area of rectangle = " << length * breadth << endl;
    }
};

int main() {
```

```
    Rectangle r;
    r.accept();
    r.displayArea();
    return 0;
}
```

Explanation: Area of a rectangle = length × breadth.

6. Bank account details

```
#include <iostream>
using namespace std;

class BankAccount {
    long long accountNo;
    double balance;
public:
    void accept() {
        cout << "Enter account number and balance: ";
        cin >> accountNo >> balance;
    }
    void display() {
        cout << "\nAccount Number: " << accountNo;
        cout << "\nBalance: " << balance << endl;
    }
};

int main() {
    BankAccount b;
    b.accept();
    b.display();
    return 0;
}
```

Explanation: The account number and balance are accepted and displayed through one BankAccount object.

7. Book details (one object)

```
#include <iostream>
using namespace std;

class Book {
    string title, author;
    float price;
public:
    void accept() {
        cout << "Enter book title, author and price: ";
        cin >> title >> author >> price;
    }
    void display() {
        cout << "\nTitle: " << title;
        cout << "\nAuthor: " << author;
        cout << "\nPrice: " << price << endl;
    }
};

int main() {
    Book b;
    b.accept();
    b.display();
    return 0;
}
```

Explanation: This program creates one Book object and displays its title, author and price.

8. Display books whose price is greater than 50 (five objects)

```
#include <iostream>
using namespace std;

class Book {
    string title, author;
    float price;
public:
    void accept() {
        cout << "Enter title, author and price: ";
        cin >> title >> author >> price;
```

```
    }
    void display() {
        cout << "\nTitle: " << title
             << "\nAuthor: " << author
             << "\nPrice: " << price << "\n";
    }
    bool isCostly() {
        return price > 50;
    }
};

int main() {
    Book b[5];

    for (int i = 0; i < 5; i++) {
        cout << "\nBook " << i + 1 << endl;
        b[i].accept();
    }

    cout << "\nBooks with price greater than 50:\n";
    for (int i = 0; i < 5; i++) {
        if (b[i].isCostly())
            b[i].display();
    }
    return 0;
}
```

Explanation: An array stores five books. The condition price > 50 selects the books to display.

9. Display the student with the highest percentage (two objects)

```
#include <iostream>
using namespace std;

class Student {
    int rollNo;
    string name;
    float m1, m2, percentage;
public:
    void accept() {
        cout << "Enter roll number and name: ";
        cin >> rollNo >> name;
        cout << "Enter marks of two subjects: ";
        cin >> m1 >> m2;
        percentage = (m1 + m2) / 2.0;
    }
    float getPercentage() {
        return percentage;
    }
    void display() {
        cout << "\nRoll No: " << rollNo;
        cout << "\nName: " << name;
        cout << "\nMarks: " << m1 << ", " << m2;
        cout << "\nPercentage: " << percentage << "%\n";
    }
};

int main() {
    Student s[2];

    for (int i = 0; i < 2; i++) {
        cout << "\nStudent " << i + 1 << endl;
        s[i].accept();
    }

    int highest = 0;
    if (s[1].getPercentage() > s[0].getPercentage())
        highest = 1;

    cout << "\nStudent with highest percentage:\n";
    s[highest].display();
    return 0;
}
```

Explanation: The percentage of both students is compared. The object with the larger percentage is displayed. Each subject is assumed to be out of 100.

10. Employee total salary (DA 10%, HRA 20%)

```
#include <iostream>
using namespace std;

class Employee {
    int empId;
    string name;
    double basic, da, hra, total;
public:
    void accept() {
        cout << "Enter employee ID and name: ";
        cin >> empId >> name;
        cout << "Enter basic salary: ";
        cin >> basic;
    }
    void calculate() {
        da = basic * 0.10;
        hra = basic * 0.20;
        total = basic + da + hra;
    }
    void display() {
        cout << "\nID: " << empId;
        cout << "\nName: " << name;
        cout << "\nBasic Salary: " << basic;
        cout << "\nDA: " << da;
        cout << "\nHRA: " << hra;
        cout << "\nTotal Salary: " << total << "\n";
    }
};

int main() {
    Employee e[2];
    for (int i = 0; i < 2; i++) {
        cout << "\nEmployee " << i + 1 << endl;
        e[i].accept();
        e[i].calculate();
    }
    for (int i = 0; i < 2; i++)
        e[i].display();
    return 0;
}
```

Explanation: DA = 10% of basic salary, HRA = 20% of basic salary, and total = basic + DA + HRA.

11. Area using method overloading

```
#include <iostream>
using namespace std;

class Shape {
public:
    double area(double r) {             // Circle
        return 3.14 * r * r;
    }
    double area(double l, double b) {   // Rectangle
        return l * b;
    }
    double area(double side, int) {     // Square
        return side * side;
    }
    double area(double base, double h, char) { // Triangle
        return 0.5 * base * h;
    }
};

int main() {
    Shape s;
    double r, l, b, side, base, h;

    cout << "Enter radius: ";
    cin >> r;
    cout << "Circle area = " << s.area(r) << endl;

    cout << "Enter rectangle length and breadth: ";
    cin >> l >> b;
    cout << "Rectangle area = " << s.area(l, b) << endl;

    cout << "Enter square side: ";
    cin >> side;
    cout << "Square area = " << s.area(side, 0) << endl;

    cout << "Enter triangle base and height: ";
```

```
    cin >> base >> h;
    cout << "Triangle area = " << s.area(base, h, 't') << endl;
    return 0;
}
```

Explanation: Method overloading means using the same function name with different parameter lists. The extra int and char parameters distinguish the square and triangle versions.

12. Mobile details

```
#include <iostream>
using namespace std;

class Mobile {
    string companyName;
    float screenSize;
public:
    void accept() {
        cout << "Enter company name and screen size: ";
        cin >> companyName >> screenSize;
    }
    void display() {
        cout << "\nCompany: " << companyName;
        cout << "\nScreen Size: " << screenSize << " inches\n";
    }
};

int main() {
    Mobile m;
    m.accept();
    m.display();
    return 0;
}
```

Explanation: The Mobile class stores company name and screen size for one object.

13. Area using constructor overloading

```
#include <iostream>
using namespace std;

class Shape {
    double result;
public:
    Shape(double r) {                 // Circle
        result = 3.14 * r * r;
        cout << "Circle area = " << result << endl;
    }
    Shape(double l, double b) {       // Rectangle
        result = l * b;
        cout << "Rectangle area = " << result << endl;
    }
    Shape(double side, int) {         // Square
        result = side * side;
        cout << "Square area = " << result << endl;
    }
    Shape(double base, double h, char) { // Triangle
        result = 0.5 * base * h;
        cout << "Triangle area = " << result << endl;
    }
};

int main() {
    double r, l, b, side, base, h;

    cout << "Enter circle radius: ";
    cin >> r;
    Shape c(r);

    cout << "Enter rectangle length and breadth: ";
    cin >> l >> b;
    Shape rec(l, b);

    cout << "Enter square side: ";
    cin >> side;
    Shape sq(side, 0);

    cout << "Enter triangle base and height: ";
    cin >> base >> h;
```

```
    Shape tri(base, h, 't');

    return 0;
}
```

Explanation: Constructor overloading means defining multiple constructors with different parameter lists. Creating each object calls the matching constructor.

14. Rectangle area using parameterized constructors (two objects)

```
#include <iostream>
using namespace std;

class Rectangle {
    float length, width;
public:
    Rectangle(float l, float w) {
        length = l;
        width = w;
    }
    void displayArea() {
        cout << "Area = " << length * width << endl;
    }
};

int main() {
    float l, w;

    cout << "Enter length and width of first rectangle: ";
    cin >> l >> w;
    Rectangle r1(l, w);

    cout << "Enter length and width of second rectangle: ";
    cin >> l >> w;
    Rectangle r2(l, w);

    cout << "\nFirst rectangle: ";
    r1.displayArea();
    cout << "Second rectangle: ";
    r2.displayArea();
    return 0;
}
```

Explanation: A parameterized constructor receives length and width when each Rectangle object is created.

15. Student activity participation using class, struct and enum

```
#include <iostream>
#include <string>
using namespace std;

enum ParticipationStatus {
    NotParticipated = 1,
    Participated,
    Completed
};

struct Activity {
    string activityName;
    int score;
};

class Student {
    string name, division;
    int rollNo;
    Activity act;
    ParticipationStatus status;
public:
    void accept() {
        cout << "Enter student name: ";
        getline(cin >> ws, name);
        cout << "Enter roll number and division: ";
        cin >> rollNo >> division;

        cout << "Enter activity name: ";
        getline(cin >> ws, act.activityName);
        cout << "Enter activity score: ";
        cin >> act.score;
```

```
        int choice;
        cout << "Status (1=Not Participated, 2=Participated, 3=Completed): ";
        cin >> choice;

        if (choice == 1 || choice == 2 || choice == 3)
            status = static_cast<ParticipationStatus>(choice);
        else
            status = NotParticipated;
    }

    void display() {
        cout << "\n--- Student Activity Summary ---\n";
        cout << "Name: " << name << "\nRoll No: " << rollNo;
        cout << "\nDivision: " << division;
        cout << "\nActivity: " << act.activityName;
        cout << "\nActivity Score: " << act.score;
        cout << "\nParticipation Status: ";

        switch (status) {
            case NotParticipated: cout << "Not Participated"; break;
            case Participated: cout << "Participated"; break;
            case Completed: cout << "Completed"; break;
        }
        cout << endl;
    }
};

int main() {
    Student s;
    s.accept();
    s.display();
    return 0;
}
```

Explanation: Activity is a struct, ParticipationStatus is an enum, and Student is a class. The switch statement prints the selected status.

16. Pointer to derived class in single and multilevel inheritance

```
#include <iostream>
using namespace std;

// Single inheritance
class Person {
public:
    void showPerson() {
        cout << "This is Person class\n";
    }
};

class Student : public Person {
public:
    void showStudent() {
        cout << "This is Student class\n";
    }
};

// Multilevel inheritance
class Marks : public Student {
public:
    void showMarks() {
        cout << "This is Marks class\n";
    }
};

int main() {
    // Pointer to derived class in single inheritance
    Student s;
    Student *p1 = &s;
    p1->showPerson();
    p1->showStudent();

    cout << "\nMultilevel inheritance:\n";
    Marks m;
    Marks *p2 = &m;
    p2->showPerson();
    p2->showStudent();
    p2->showMarks();

    return 0;
}
```

Explanation: A derived-class pointer points to an object of that derived class. It can access inherited public members and its own public members.

17. Diamond problem and solution using virtual base class

```
#include <iostream>
using namespace std;

class A {
public:
    int value;
    A() {
        value = 10;
    }
};

class B : virtual public A { };
class C : virtual public A { };

class D : public B, public C { };

int main() {
    D obj;
    obj.value = 50;
    cout << "Value = " << obj.value << endl;
    return 0;
}
```

Explanation: Without virtual inheritance, D would receive two separate copies of A (one through B and one through C), causing ambiguity. The keyword virtual ensures D has only one shared A base-class subobject.

18. Function template to swap integers and floats

```
#include <iostream>
using namespace std;

template <class T>
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    float p = 1.5, q = 2.5;

    cout << "Before integer swap: " << x << " " << y << endl;
    swapValues(x, y);
    cout << "After integer swap: " << x << " " << y << endl;

    cout << "\nBefore float swap: " << p << " " << q << endl;
    swapValues(p, q);
    cout << "After float swap: " << p << " " << q << endl;

    return 0;
}
```

Explanation: A function template works with different data types. T becomes int for the first call and float for the second call.

19. Server and Client inheritance

```
#include <iostream>
using namespace std;

class Server {
protected:
    string serv_name;
    int serv_port;
public:
    void show() {
        cout << "\nServer Name: " << serv_name;
        cout << "\nServer Port: " << serv_port << endl;
    }
};
```

```
class Client : public Server {
    string client_name;
    int client_port;
public:
    void accept() {
        cout << "Enter server name and port: ";
        cin >> serv_name >> serv_port;
        cout << "Enter client name and port: ";
        cin >> client_name >> client_port;
    }
    void show() {
        Server::show();
        cout << "Client Name: " << client_name;
        cout << "\nClient Port: " << client_port << endl;
    }
};

int main() {
    Client c;
    c.accept();
    c.show();
    return 0;
}
```

Explanation: Client publicly inherits Server. The derived class accepts both sets of details and calls Server::show() to display the server details.

20. File creation, writing, closing and reading

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream outFile("student.txt");

    if (!outFile) {
        cout << "Unable to create file.";
        return 1;
    }

    outFile << "Welcome to C++ File Handling\n";
    outFile << "This information is written into a file.\n";
    outFile.close();

    ifstream inFile("student.txt");

    if (!inFile) {
        cout << "Unable to open file.";
        return 1;
    }

    string line;
    cout << "File contents:\n";
    while (getline(inFile, line)) {
        cout << line << endl;
    }

    inFile.close();
    return 0;
}
```

Explanation: ofstream creates and writes to the file. close() closes it. ifstream opens it again, and getline() reads it line by line. The file is created in the program's working directory.

21. Multilevel inheritance: Student, Marks and Result

```
#include <iostream>
using namespace std;

class Student {
protected:
    int rollNo;
    string name;
public:
    void acceptStudent() {
        cout << "Enter roll number and name: ";
```

```
        cin >> rollNo >> name;
    }
};

class Marks : public Student {
protected:
    int m1, m2, m3;
public:
    void acceptMarks() {
        cout << "Enter marks of three subjects: ";
        cin >> m1 >> m2 >> m3;
    }
};

class Result : public Marks {
public:
    void display() {
        int total = m1 + m2 + m3;
        cout << "\nRoll No: " << rollNo;
        cout << "\nName: " << name;
        cout << "\nMarks: " << m1 << " " << m2 << " " << m3;
        cout << "\nTotal: " << total << endl;
    }
};

int main() {
    Result r;
    r.acceptStudent();
    r.acceptMarks();
    r.display();
    return 0;
}
```

Explanation: Student is the base class, Marks inherits Student, and Result inherits Marks. Three subjects are used because the question does not specify a number of subjects.

22. Single inheritance: Person and Student

```
#include <iostream>
using namespace std;

class Person {
protected:
    string name;
    int age;
public:
    void acceptPerson() {
        cout << "Enter name and age: ";
        cin >> name >> age;
    }
};

class Student : public Person {
    int rollNo;
public:
    void accept() {
        acceptPerson();
        cout << "Enter roll number: ";
        cin >> rollNo;
    }
    void display() {
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
        cout << "\nRoll Number: " << rollNo << endl;
    }
};

int main() {
    Student s;
    s.accept();
    s.display();
    return 0;
}
```

Explanation: Student inherits name and age from Person and adds its own roll number. One Student object displays all details.